

twyla

A static-site generator whose only authoring language is typst.

Twyla embeds the typst compiler as a Rust library (the typst crate), not the CLI, so it can route typst files to URLs, intercept asset resolution to fingerprint/optimize images, walk the compiled document for metadata, emit RSS/Atom, and (eventually) do incremental rebuilds.

The driving target is porting samsartor.com (currently zola + tera + markdown, at ~/Src/site) to twyla + typst, with HTML AST equivalence as the porting harness.

Why pure typst (no template layer)

Considered: a tera-like template engine layered on top of typst, by analogy with zola layering tera on top of markdown. Rejected — typst already provides the moving parts a template language would re-invent:

- **Inheritance** — `base.html + {% block %}` becomes a typst function taking a body parameter, with `show/set` rules supplying the defaults.
- **Filters** — typst functions compose better than pipeline syntax.
- **Shortcodes** — already just typst functions, no separate registry.
- **HTML output** — `html.elem(..)` and the typed `html.div(..)` family let us emit arbitrary tag + class + attribute structures.

Cost of **adding** a template language: two scoping models, two escape hatches, two contexts for the author to switch between. Cost of **not** adding one: typst's HTML export is younger than tera, so ergonomic rough edges land on us. That's where twyla earns its keep — wrapping the crate gives us room to shape output rather than living with defaults.

Architecture

Routing & bundle

`#document(path, ..)` (typst PR 7964) is the routing primitive: one call per emitted HTML file. Cross-document links use labels; typst computes relative paths automatically. `#asset(path, bytes)` writes raw files into the same bundle output.

Twyla's job, eventually: scan `content/*.typ`, generate a `main.typ` in memory that emits one `#document(..)` per source file (path derived from filesystem layout with `metadata()` override), compile the bundle, post-process. Today the render binary handles a single user-supplied entrypoint at a time, no scan-and-generate yet.

Asset model

Two orthogonal axes for asset primitives — **input source** and **output kind**:

	Output: bytes	Output: url
Input: path	<code>read(path) -> bytes</code> <i>Already typst's built-in.</i>	<code>asset-url(path) -> str</code> — emit URL pointing at file. Eventually fingerprinted.
Input: bytes	<i>byte transforms (font rewrite, webp encode); rust or typst.</i>	<code>content-url(bytes, name) -> str</code> — hash, write to bundle, URL.
Input: content (<i>typst Content</i>)	<code>render(content, format) -> bytes</code> — sub-compile. Overlaps with typst's <code>#document(path,..)</code> .	<code>asset.from-content(...).url</code> — render + register + URL.

Principle: byte transforms run in Rust (or in user typst where ergonomic); presentation decisions (`inline <svg>` vs `<img>` vs `<picture>` vs `<use href>` vs `iframe`) run in user typst templates. URL-returning primitives are the seam: above them typst owns layout and emission; below them Rust owns paths, fingerprinting, and the on-disk bundle.

Phase 1 implements only read (free from typst). URL primitives are deferred until a second use case forces the API surface.

#document is the page-level expression of “content input → URL.” Open question whether the eventual asset.from-content(..) API subsumes it, or whether they stay separate (pages routed by content layout vs assets routed by hash).

Resolution pass

Twyla owns a post-compile **resolution pass** — a walk over the bundle’s serialized output that finds placeholder markers and substitutes their final form. Inspired by the future/promise pattern: typst emits a deferred value; Rust resolves it later. This is the seam between “things typst can express” and “things we want in the output.”

Phase 1 has one placeholder: **raw HTML**. Typst has no `html.raw`; only `<script>` and `<style>` bodies are emitted unescaped (verified: `typst-html::tag::is_raw`). We exploit that: user typst calls `raw-html(content)` which emits `<script type="x-twyla-raw-html">..bytes..</script>`; the resolution pass replaces each match with the inner content spliced inline. Used by the `svg/diagram` shortcodes to embed external SVG files.

Anticipated placeholders, each earned by a use case:

- **Image optimization** — `<twyla-image data-src=".." data-sizes="..">` resolves to `<picture>/<img srcset>` after webp/avif encoding.
- **Asset-as-content** — sub-compile registered typst content to SVG, splice inline.
- **Feed expansion** — `<twyla-feed-entries data-query="..">` resolves to N siblings driven by introspector query results.

Implementation strategy: keep the `str-replace` pattern until we have ≥ 2 marker kinds, then generalize to a typed handler registry.

The resolution pass is **the** reason twyla doesn’t need to fork typst — it’s pure post-processing of typst output, no internals touched.

Tooling

Porting binaries

Three CLI binaries + a shell driver, all in this repo:

- **twyla-render-page** `--root <dir>` `<entrypoint.typ>` — compile as bundle, run resolution pass, print the single document’s HTML.
- **twyla-extract** `<selector>` `<input.html>` — print inner HTML of the first matching element. Selectors: `class:<name>`, `tag:<name>`.
- **twyla-diff** `--textonly-pre` `--ignore-attr <tag>:<attr>`... `<expected>` `<actual>` — structural AST diff with optional porting relaxations.
- **port-page.sh** — render + extract + diff for a single page. Hardcoded to `guis-2`; generalize when we port the second page.

AST diff harness (`twyla::diff`)

Library + CLI. Parses both sides via `html5ever`, normalizes (lowercase tags/attrs, sort attrs and class tokens, collapse inter-block whitespace, preserve `<pre>/<code>/<script>/<style>/<textarea>` text verbatim, drop comments), walks both trees in lockstep, reports the first structural divergence.

Relaxations are opt-in (`Matcher`, `RelaxationRule`) pairs, first-match wins. Defaults to zero — porting starts strict.

Matcher	Matches
<code>Tag("p")</code>	every <code><p></code> element

Matcher	Matches
TagAttr { tag, attr, value }	matching tag with attr == value
TagAttrExists { tag, attr }	matching tag with attr set

RelaxationRule	Effect
IgnoreEntirely	subtree considered equal regardless
IgnoreAttribute(name)	skip one attribute on matched element
TextOnly	compare concatenated text only; ignore structure

Standing rule: every new relaxation is a place twyla and zola diverge. Run them by Sam first.

Deliberately deferred: per-section relaxations (different rules under different ancestors), multi-divergence reporting (today: bails on first divergence), attribute-value matchers (“ignore style only when value = color: red”).

Status

1. ~~HTML-export prototype~~ — done (experiment 1).
2. ~~Multi-document bundle output~~ — done (experiment 2).
3. ~~AST-diff harness~~ — done (twyla::diff).
4. ~~Port one page, body only (guis-2)~~ — done. port-page.sh matches under two relaxations: --textonly-pre (code blocks) and --ignore-attr td:style --ignore-attr th:style (typst tables don’t emit alignment as CSS).
5. **Port guis-2 full page** — base + page templates. Next.
6. **Generalize** — routing, asset pipeline, feed, dev server. After.

Roadmap

Working assumption: walk from “one page, body” outward, adding twyla infrastructure only when a porting case forces it.

Near term — finish guis-2

1. **Page templates as typst functions.** Port base.html + page.html to a typst module the user imports. The function takes title, description, body, and any page-extra fields as parameters, emits via #show: page-template.with(..). Open: does twyla auto-apply templates via its generated main.typ (saves the user one line per page) or do we ask users to #show: ... themselves (more honest, less magic)? Resolve by writing the explicit version first; revisit when the boilerplate-per-page hurts.
2. **Second page.** Port guis-1 or guis-3 to validate the workflow generalizes. Will surface what was guis-2-specific in port-page.sh/shortcodes.typ — drives the first round of generalization.

Medium term — twyla becomes an SSG

1. **Multi-entriypoint routing.** Replace the one-#document-per-render-call constraint with twyla scanning content/*.typ, generating an in-memory main.typ that emits one #document(..) per source. Path derives from filesystem layout, with per-page metadata() override.
2. **Real asset-url primitive.** Drop the hardcoded base_url in templates/shortcodes.typ. Comes from a twyla config layer (TOML or typst-side). This is the moment we promote our one resolution-pass placeholder into a registry, because the URL primitive needs hooks for fingerprinting/dedup.
3. **Image optimization pipeline.** Webp/avif encoding, srcset generation, in Rust as a resolution-pass handler. User typst calls one function; the registered handler does the byte transforms and emits the resolved <picture>.
4. **Feed generation.** #document("atom.xml", ..) in twyla’s library, querying bundle.introspector for kind: "post" entries.

Longer term

1. **Dev server with live reload.** Open question: do we own it (twyla watches sources, serves with WS reload script) or shell out to vite dev for now and revisit?
2. **Incremental rebuilds.** Hold one world across watch cycles; lean on comemo memoization. Revisit if cold-start becomes a perf wall.
3. **Sass / JS bundling.** Probably shell out (vite or esbuild) rather than reimplement. Twyla orchestrates, doesn't compile.

Validation

1. **EXAMPLES-corpus test.** Run a port-page-style diff against representative pages from [getzola's EXAMPLES](#). Catches feature breadth the personal site doesn't exercise. Should produce a corpus of (zola-html, twyla-html, relaxations) triples that we keep green on CI.

Lessons (from porting guis-2 body)

Compact gotcha list; details in commits.

- **Smart quotes** — leave typst's smartquote on; matches zola's smart_punctuation = true character-for-character.
- **Show rules fire in HTML mode** (verified de6f400). Folk claim otherwise; not true for content-transformation rules. Layout-specific rules (positioning, frame production) may not, untested.
- **Pulldown auto-wraps inline content in <p> inside containers** (<div>, <blockquote>, <td>); typst's HTML export doesn't. Wrap manually with #html.p[...] until we find a cleaner pattern.
- **Native #table** works for HTML. Only divergence: align: (..) doesn't reflect into per-cell style="text-align:.." (typst treats it as layout-only). Relax --ignore-attr td:style --ignore-attr th:style to absorb.
- **Show rules can't replace typst's auto-generated wrapper.** A show table.cell.where(x: 0): it => html.elem("td", ..) produces <td><td ..>..</td></td>. Same hazard expected on list.item, enum.item, etc.
- **html.a(rel: ..)** takes an array of enum tokens, not a space-joined string: rel: ("noopener", "external").
- **External-link auto-attrs** (rel="noopener external" target="_blank") via show link: it => ... checking type(it.dest) == str and prefix.
- **html.elem for non-typed attrs.** html.div doesn't expose align; use html.elem("div", attrs: (align: "center"), ..).
- **Inline SVG via the resolution pass.** <script type="x-twyla-raw-html"> falls into RawMode::Keep because the type attribute isn't text/javascript.
- **Typst #let chain-across-newlines hazard.** #let f(s) = s\n .replace(..) parses as identity-of-s with the dot-calls as orphan markup, no error. Wrap multi-line chains in (...).

Upstream-watch list

Worth raising upstream (or watching for) on typst:

1. html.raw(content) — first-class verbatim HTML. Today the resolution pass abuses <script>. Likely controversial (typst values structured output) but every HTML consumer hits this.
2. #table(align: ..) reflecting into style="text-align: .." on <th>/<td> in HTML mode. Same for colspan/rowspan attributes.
3. Show rules on table.cell/list.item/enum.item that can **replace** typst's auto-generated wrapper in HTML, not just decorate its body.
4. Hook on image() to emit external src instead of data URLs in HTML mode — would benefit anyone doing HTML export at scale.
5. Per-document defaults override (e.g. document-level smartquote(enabled: false) declared once).

6. Introspector exposure of cross-document link resolution data — useful for fingerprint-then-rewrite passes.

Fork vs library

Standing assumption: **do not fork typst**. Twyla is a Rust binary that consumes typst as a library, supplies a custom `World`, ships a small typst library of helpers, and post-processes the bundle. If something is genuinely better as an upstream change, **upstream it** before forking. Fork threshold: library + upstreaming both proven inadequate.

Why: typst’s HTML/bundle work is actively evolving in main. A fork puts us permanently downstream of grammar, library, and HTML-output changes — a real maintenance tax paid forever for benefits mostly reachable from outside.

Decisions to date (every feature considered has landed on “library”): asset fingerprinting (post-export DOM walk), image optimization (twyla function bypassing `image()`), pretty URLs (`#document(..)` paths), path computation, hot reload (WebSocket-inject during dev), RSS/sitemap, per-document `smartquote/lang` (already configurable), incremental compile (comemo), file mtime via `sys.inputs`, `async/HTTP` at compile (pre-fetch in Rust). No fork: HTML-syntax-mode (the JSX-style sugar isn’t worth shadowing typst’s grammar).

If we ever hit a real wall, document what we tried and why it didn’t work — that’s the evidence base for reconsidering.

Notes

- VCS: both `~/Src/twyla` and `~/Src/site` are managed with `jj`. **Rule:** always `jj new` before editing — `jj` has no staging area, so edits land in whatever revision the working copy points at.
- Typst pinned to main `de6f400` (2026-04-11) for the bundle export feature (PR 7964). Switch back to `crates.io` once `0.15.x` ships.
- Tera shortcodes in scope for porting: `centered`, `diagram`, `image`, `math`, `svg`, `tagline`, `var` (`~/Src/site/templates/shortcodes/`). Phase 1 covered the first four; `math/tagline/var` come with pages that use them.
- Tera templates in scope: `base.html`, `page.html`, `section.html`, `index.html`, `components.html`, `404.html`, `atom.xml`, `paperpillls.html`.
- Known porting hazards: ROT13-encoded email obfuscation in footer, conditional asset loading (`page.extra.tilings`), SVG inlining with font-family rewriting (handled by the `svg/diagram shortcodes`).